Fl.Event.Handling

This library provides the core abstractions and implementations for event handling using a functional approach built on top of LanguageExt.

Abstractions

`IEventHandler<TEvent>`

Represents a single handler responsible for reacting to an event of type `TEvent`.

```
public interface IEventHandler<TEvent> where TEvent : class
{
    EitherAsync<Error, Unit> HandleAsync(TEvent evt);
}
```

Implement this interface to define the logic that should execute when a specific event is received. Multiple implementations can be registered for the same `TEvent` and composed through an `IEventProcessor<TEvent>`.

`IEventProcessor<TEvent>`

Represents an orchestrator that coordinates one or more `IEventHandler<TEvent>` instances to process an event.

```
public interface IEventProcessor<TEvent> where TEvent : class
{
    EitherAsync<Error, Unit> ProcessAsync(TEvent evt);
}
```

Both methods return `EitherAsync<Error, Unit>`: an asynchronous computation that yields `Unit` on success or an `Error` on failure, keeping the processing pipeline fully functional and free of thrown exceptions.

Processors

`SequentialEventProcessor<T>`

Dispatches an event to all registered `IEventHandler<T>` instances **one at a time**, in registration order. If any handler returns an `Error`, processing stops immediately and the error is propagated.

```
var processor = new SequentialEventProcessor<MyEvent>(handlers);
EitherAsync<Error, Unit> result = processor.ProcessAsync(myEvent);
```

`ParallelEventProcessor<T>`

Dispatches an event to all registered `IEventHandler<T>` instances **concurrently**, awaiting all of them before returning. Use this when handlers are independent and latency matters.

```
var processor = new ParallelEventProcessor<MyEvent>(handlers);
EitherAsync<Error, Unit> result = processor.ProcessAsync(myEvent);
```

Decorators

Both decorators implement `IEventProcessor<TEvent>` and wrap an existing processor, following the decorator pattern so they can be freely composed.

`ExceptionCatcherEventProcessor<TEvent>`

Wraps an inner processor and converts any unhandled exception thrown during processing into an `Error` value, preventing exceptions from escaping the functional pipeline.

```
IEventProcessor<MyEvent> safe = new ExceptionCatcherEventProcessor<MyEvent>
(innerProcessor);
```

`LoggingEventProcessor<TEvent>`

Wraps an inner processor and logs the outcome of each processing call using a Serilog `ILogger`.

```
IEventProcessor<MyEvent> logged = new LoggingEventProcessor<MyEvent>
(innerProcessor, logger);
```

Composition example

Decorators can be stacked to combine concerns:

```
IEventProcessor<MyEvent> processor =
    new LoggingEventProcessor<MyEvent>(
```

```
        new ExceptionCatcherEventProcessor<MyEvent>(
            new SequentialEventProcessor<MyEvent>(handlers)),
        logger);
```

This creates a processor that handles events sequentially, catches any exceptions, and logs every outcome.

Namespace Fl.Event.Handling

Classes

ExceptionCatcherEventProcessor<TEvent>

An IEventProcessor<TEvent> decorator that catches any unhandled exceptions thrown by the inner processor and converts them into LanguageExt.Common.Error values.

LoggingEventProcessor<TEvent>

An IEventProcessor<TEvent> decorator that adds structured logging around the inner processor's execution using a Serilog Serilog.ILogger.

ParallelEventProcessor<T>

An IEventProcessor<TEvent> that dispatches an event to all registered IEventHandler<TEvent> instances concurrently.

SequentialEventProcessor<T>

An IEventProcessor<TEvent> that dispatches an event to all registered IEventHandler<TEvent> instances one at a time, in order.

Interfaces

IEventHandler<TEvent>

Defines a handler responsible for processing a single event of type `TEvent`.

IEventProcessor<TEvent>

Defines a processor that orchestrates the handling of events of type `TEvent`.